

第 44 讲：Dropout

训练时为什么要故意随机“关掉”一部分神经元？

《面向 LLM / Agent Systems 科研的深度学习零基础课》

【科研洞察】

第 43 讲已经建立了 Regularization 的统一视角：训练目标不是一味把 Training Loss 压到最低，而是要限制模型利用训练集中的偶然模式。Weight Decay 是通过约束 Parameter 尺度来做这件事；本讲的 Dropout 则采用另一条路线：训练时主动给网络制造随机缺失，迫使模型不要长期依赖某一小撮固定神经元或固定路径。

一、这一讲到底要解决什么问题？

假设一个 MLP 的隐藏层有很多神经元。训练集很小时，网络可能形成一种脆弱的“内部合作方式”：

- 某几个神经元总是一起出现、互相补偿；
- 某条固定路径非常依赖训练集中的偶然细节；
- 只要这条路径在训练集上有效，Training Loss 就能很低；
- 但换到新数据，这种依赖可能立刻失效。

Dropout 的想法非常直接：

训练时随机让一部分激活暂时变成 0。

网络每次 Forward 都面对一个略微不同的“临时子网络”，因此很难永远依赖同一组内部通道。

【原理】

Dropout 是一种**随机正则化**。它的核心不是“删掉 Parameter”，也不是“永久删除神经元”，而是在**训练时随机屏蔽激活值 (activation)**，并在评估 / 推理时关闭这种随机性。

二、先看最小例子：一个向量经过 Dropout 会发生什么？

假设隐藏层激活：

$$x = [1, 2, 3, 4]$$

Dropout probability：

$$p = 0.5$$

意思是：每个元素在训练时有 50% 概率被丢弃。

某一次随机采样得到 Mask：

$$m = [1, 0, 1, 0]$$

如果只是最朴素地相乘：

$$m \odot x = [1, 0, 3, 0]$$

均值会系统性降低。原始平均值：

$$\frac{1 + 2 + 3 + 4}{4} = 2.5$$

而大量随机丢弃以后，平均尺度大约只剩一半。

所以 PyTorch 的 `nn.Dropout` 默认采用 **Inverted Dropout**：训练阶段不仅随机置零，还把保留下来的元素除以 keep probability。

三、公式：Dropout 到底在计算什么？

定义：

$$p = \text{drop probability}$$

$$q = 1 - p = \text{keep probability}$$

对第 i 个激活：

$$m_i \sim \text{Bernoulli}(q)$$

也就是：

$$m_i = \begin{cases} 1, & \text{概率 } q \\ 0, & \text{概率 } p \end{cases}$$

训练模式下：

$$y_i = \frac{m_i}{q} x_i$$

评估模式下：

$$y_i = x_i$$

公式逐项拆解

- x_i ：Dropout 之前的激活；
- m_i ：随机 Mask；
- p ：丢弃概率；
- $q = 1 - p$ ：保留概率；
- $1/q$ ：训练阶段的缩放因子；
- y_i ：Dropout 之后的激活。

【具体例子】

若 $p = 0.5$ ，则 $q = 0.5$ ，保留下来的元素会乘：

$$\frac{1}{q} = 2$$

因此输入 1 在训练模式下不是 “0 或 1”，而是 “0 或 2”。

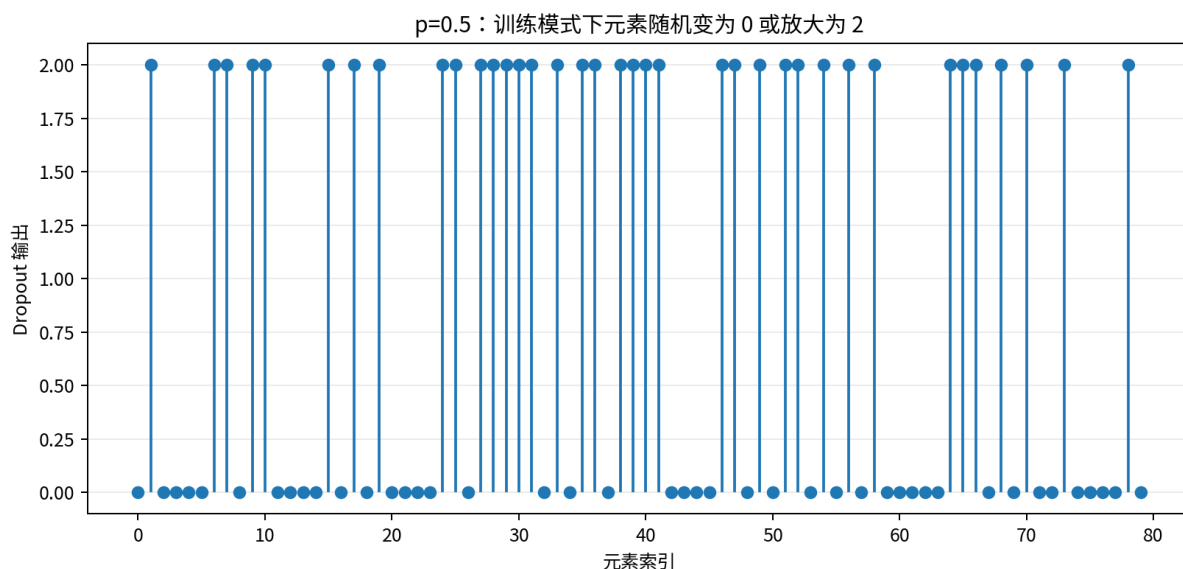


图 1: 实验: $p = 0.5$ 时, 训练态激活随机变成 0 或被放大为原来的 2 倍。

四、为什么训练时要除以 q ? ——看期望值

训练时:

$$y_i = \frac{m_i}{q} x_i$$

因为:

$$E[m_i] = q$$

所以:

$$E[y_i] = E\left[\frac{m_i}{q} x_i\right] = \frac{x_i}{q} E[m_i] = x_i$$

即:

$$E[y_i] = x_i$$

这意味着: 虽然每次 Forward 的结果随机变化, 但从很多次采样平均来看, Activation 的期望尺度保持不变。

本讲实验令:

$$x = [1, 2, 4, 8], \quad p = 0.3$$

重复 20000 次 Dropout 后, 经验均值约为:

$$[1.0122, 1.9954, 3.9937, 8.0297]$$

已经非常接近原输入。

【科研洞察】

Dropout 的本质不是“让激活整体变小”, 而是在保持期望尺度基本不变的同时, 注入随机噪声与结构扰动。这正是其 Regularization 作用的来源之一。

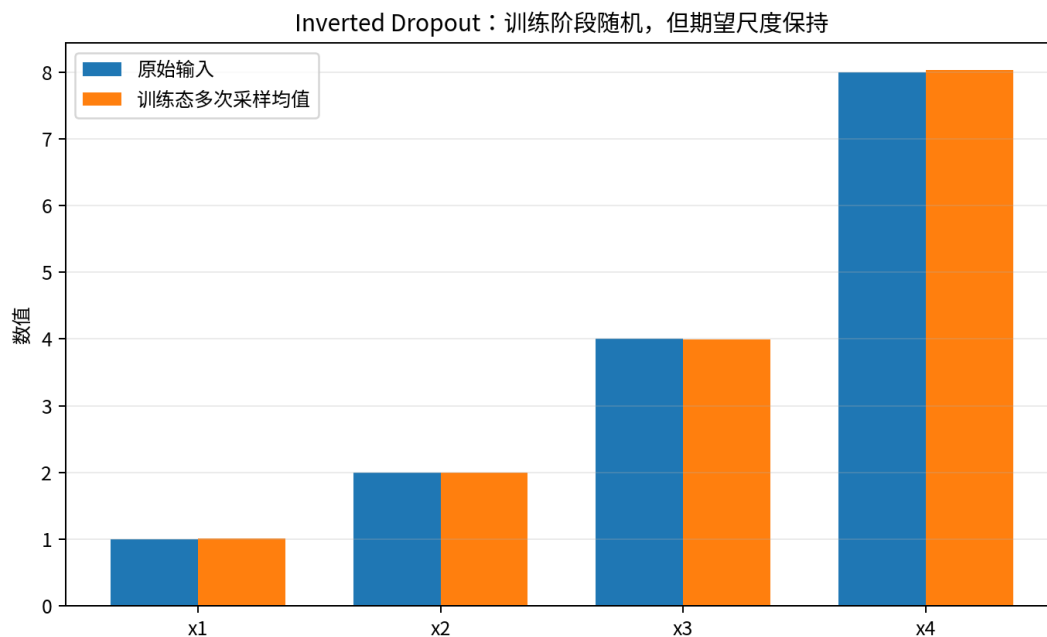


图 2: Inverted Dropout 保持期望尺度：单次 Forward 随机，但多次平均接近原输入。

五、Dropout 保持均值，但会增加方差

对固定的 x_i ，可以进一步得到：

$$\text{Var}(y_i) = \frac{p}{q} x_i^2$$

这条公式不要求背，但直觉非常重要：

- p 越大，随机性越强；
- q 越小，保留元素的放大倍数越大；
- 大 Activation 在 Dropout 下也会产生更大的随机波动。

所以 Dropout probability 不是一个“越大越正则”的无脑旋钮。正则太弱可能压不住过拟合；太强则会让网络训练时信号过于破碎，最终欠拟合。

六、Dropout 改不改变 Shape？有没有 Parameter？

假设：

$$X.shape = (B, T, D)$$

例如：

$$X.shape = (2, 3, 8)$$

经过：

$$Y = \text{Dropout}(X)$$

仍然：

$$Y.shape = (2, 3, 8)$$

Dropout 一般是逐元素随机置零，因此它改变的是**数值与随机路径**，不是 Tensor Shape。

并且：

<i>Dropout</i>	<i>Parameter</i>
----------------	------------------

PyTorch 中：

```

1 import torch
2 import torch.nn as nn
3
4 # 创建 Dropout; p=0.25 表示训练时每个元素有 25% 概率被丢弃
5 dropout = nn.Dropout(p=0.25)
6
7 # 模拟一个 LLM 风格的隐藏状态: [batch, token, hidden]
8 X = torch.randn(2, 3, 8)
9
10 # 训练模式下执行 Dropout
11 Y = dropout(X)
12
13 print(X.shape) # torch.Size([2, 3, 8])
14 print(Y.shape) # torch.Size([2, 3, 8])
15
16 # Dropout 没有 Weight / Bias
17 print(sum(p.numel() for p in dropout.parameters())) # 0

```

七、最容易踩坑的一组概念：train() / eval() / no_grad()

这三个东西经常一起出现，但它们控制的不是同一件事。

1. model.train()

把模型切到训练模式。对 Dropout：

<i>Mask</i>

2. model.eval()

把模型切到评估模式。对 Dropout：

<i>Mask</i>	<i>Dropout</i>
-------------	----------------

3. torch.no_grad()

控制的是：

<i>Autograd</i>

它不会自动把 Dropout 切到 eval 行为。

设置	Dropout 随机性	Autograd
train() + 正常执行	开	开
train() + no_grad()	开	关
eval() + 正常执行	关	开
eval() + no_grad()	关	关

实验中同一个输入：

```
train + grad      -> [[0, 2, 0, 0]]
train + no_grad   -> [[0, 2, 0, 0]]
eval  + grad      -> [[1, 1, 1, 1]]
eval  + no_grad   -> [[1, 1, 1, 1]]
```

说明 no_grad() 并不会关闭 Dropout。

【常见误区 / 风险】

评估 Validation / Test 时只写 torch.no_grad() 而忘记 model.eval(), 可能让 Dropout 仍保持随机, 从而使评估指标波动、偏离真正的推理行为。

八、为什么 Dropout 可以帮助 Generalization ?

可以从几个互补视角理解。

视角 1：减少对固定内部路径的依赖

没有 Dropout 时，一个神经元可能长期依赖另一个神经元“帮它补偿”。Dropout 让这种固定合作关系在每个 Step 都可能被打断，因此网络被迫学习更分散、更冗余的表示。

视角 2：训练时不断注入随机扰动

每个 Step 的 Forward Function 都略有不同：

$$f_{\theta,m}(x)$$

其中 m 是随机 Mask。因此网络不能只在一个完全固定的内部计算路径上优化。

视角 3：近似许多子网络共享 Parameter

可以把不同 Mask 看成从同一个大网络中采样不同的临时子网络。**这是一种有用直觉，而不是说推理时真的显式训练并平均了指数数量的独立模型。**

【常见误区 / 风险】

不要把“Dropout 类似 ensemble”理解成严格等价。它们共享 Parameter、训练过程强耦合，推理时通常也不是显式跑很多独立模型再平均。

九、Dropout probability p 太小、合适、太大会怎样？

本讲用一个小样本、高容量 MLP 做实验，只改变 Dropout probability：

p	Train MSE	Val MSE	Test MSE	Best Step
0.0	0.0714	0.0913	0.0927	81
0.1	0.0878	0.0893	0.0913	131
0.3	0.0682	0.0911	0.0946	2731
0.6	0.1287	0.1198	0.1241	1461

这个特定实验中 $p = 0.1$ 的 Validation 最好，但改善并不巨大。这个结果反而很适合科研训练：

- Dropout 不是必然带来巨大收益；
- 最优 p 依赖数据规模、模型容量、噪声、优化器和训练预算；
- $p = 0.6$ 已经明显过强，Train / Val / Test 都变差；
- 结论应该通过 Validation 选择，再在独立 Test 上确认。

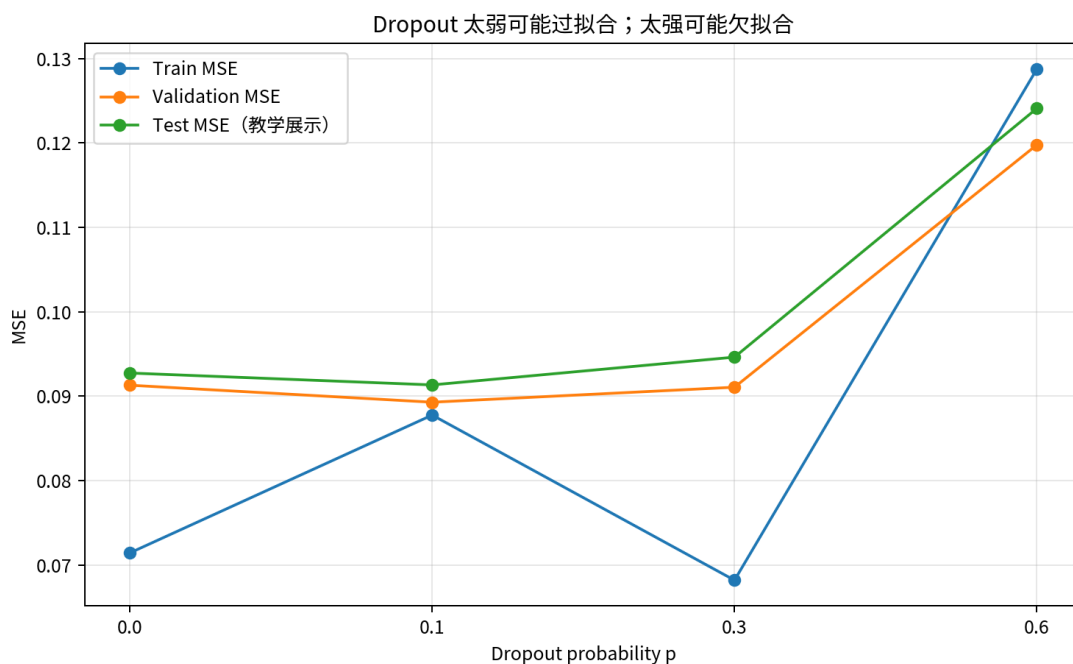


图 3: 同一模型、同一初始化和训练预算下，仅改变 Dropout probability。

十、Dropout 和 Weight Decay 到底有什么不同？

比较点	Weight Decay	Dropout
主要作用对象	Parameter	Activation / 计算路径
是否有随机性	通常没有	有
是否修改 Objective / Update	通过 Parameter penalty / decay	通过训练时随机 Mask
推理阶段	Parameter 已经被训练影响	通常关闭 Dropout
核心超参数	weight decay coefficient	drop probability p
常见风险	太大导致欠拟合	太大导致信号过度破坏

二者不是互斥的。一个模型完全可以同时使用 Weight Decay 和 Dropout，但是否需要、强度多少必须通过实验决定。

十一、PyTorch 中怎样正确写 Dropout？

一个最基本的 MLP：

```
1 import torch
2 import torch.nn as nn
3
4 class MLP(nn.Module):
5     def __init__(self):
6         super().__init__()
7         self.net = nn.Sequential(
8             nn.Linear(100, 256),
9             nn.ReLU(),
10            nn.Dropout(p=0.2), # 训练时随机丢弃 20% 的激活
11            nn.Linear(256, 10),
12        )
13
14    def forward(self, x):
15        return self.net(x)
16
17 model = MLP()
18
19 # 训练阶段：Dropout 开启
20 model.train()
21 train_logits = model(x_train)
22
23 # 评估阶段：Dropout 关闭；同时关闭 Autograd 记录
24 model.eval()
25 with torch.no_grad():
26     val_logits = model(x_val)
```

【原理】

`nn.Dropout(p)` 的 p 是**丢弃概率**，不是保留概率。 $p = 0.2$ 表示丢 20%，保留 80%。

十二、Dropout 在 Transformer / LLM 里出现在哪里？

在 Transformer 系统中，Dropout 可以出现在不同位置，例如：

- Embedding / hidden state 之后；
- Attention probability 或 Attention output 附近；
- FFN / MLP 内部或输出后；
- Residual branch 的输出路径。

这些位置的语义并不完全相同：

- 对 hidden activation 做 Dropout：随机屏蔽表示维度中的部分激活；
- 对 attention weights 做 Dropout：随机扰动“本次要从哪些 Token 汇聚信息”；
- 对 residual branch 输出做 Dropout：扰动某一子层写回 residual stream 的贡献。

【常见误区 / 风险】

不要把“Transformer 有 Dropout”理解成所有现代 LLM 都必须使用较大的 Dropout。模型规模、数据规模和训练 recipe 不同，实际配置可能很小甚至为 0。科研时应读取具体模型配置，而不是凭架构名称猜测。

十三、LLM 里的 Attention Dropout 和普通 Dropout 有什么关系？

普通 hidden dropout 可以写成：

$$Y = \text{Dropout}(X)$$

Attention 中先得到：

$$A = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

然后某些实现会在训练时：

$$\tilde{A} = \text{Dropout}(A)$$

再：

$$O = \tilde{A}V$$

此时随机屏蔽的对象不再只是普通 hidden feature，而是 Attention 的连接权重。底层 Dropout 机制仍然是随机 Mask + inverted scaling，但它发生在不同语义对象上，因此对网络行为的影响不同。

十四、Dropout 与 BatchNorm / LayerNorm：不要混淆 train/eval 行为

后面我们会学习 Normalization。这里先建立边界：

- Dropout 的 train/eval 差异主要是“随机 Mask 开 / 关”；
- BatchNorm 的 train/eval 差异还涉及 batch statistics 与 running statistics；
- LayerNorm 通常没有这种依赖 batch statistics 的 train/eval 差异。

所以：

`model.eval()` 不是“模型停止计算”，而是在告诉某些 Module：现在请采用评估阶段的行为规则。

十五、一个经常被忽略的实验细节：Validation 时一定要用 eval 模式

本讲泛化实验在每次评估 Validation 时都执行：

```
1 model.eval()
2 with torch.no_grad():
3     val_pred = model(X_val)
4     val_loss = loss_fn(val_pred, y_val)
```

如果忘记 `model.eval()`，同一个 Validation sample 多跑几次可能得到不同输出，导致：

- Validation Loss 波动；
- Best Checkpoint 选择不稳定；
- 不同实验之间比较噪声增大；
- 最终论文结果难复现。

十六、Dropout 的 Backward：被丢掉的激活会收到什么 Gradient ？

前面已经学过 Computational Graph 与 Backpropagation。Dropout 也只是计算图里的一个节点，因此它同样有自己的局部 backward 规则。

训练时：

$$y_i = \frac{m_i}{q} x_i$$

把随机 Mask m_i 看成这一次 Forward 已经确定的常量，那么局部导数：

$$\frac{\partial y_i}{\partial x_i} = \frac{m_i}{q}$$

所以上游梯度 $g_i = \partial L / \partial y_i$ 传回来时：

$$\frac{\partial L}{\partial x_i} = \frac{m_i}{q} g_i$$

这意味着：

- 若这一元素本次被丢弃， $m_i = 0$ ，则该路径本次 Gradient 也是 0；

- 若被保留, Gradient 同样会乘 $1/q$;
- 下一次 Forward 会重新采样 Mask, 所以同一个神经元下次可能又重新参与训练。

【具体例子】

设 $p = 0.5$, $q = 0.5$, 某次 Mask 为:

$$m = [1, 0, 1]$$

上游梯度:

$$g = [2, 3, 4]$$

那么传给 Dropout 输入的梯度:

$$\frac{m}{q} \odot g = [2, 0, 2] \odot [2, 3, 4] = [4, 0, 8]$$

第二个位置这一轮不仅 Forward 输出为 0, Backward 也不会沿该位置继续传播 Gradient。

这让 Dropout 和第 24–25 讲的 ReLU gate 产生一个有趣联系: 二者都可能让某条路径的局部 Gradient 变成 0, 但原因不同。ReLU 由输入符号决定; Dropout 则由随机 Mask 决定。

十七、完整执行轨迹: $\text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Dropout} \rightarrow \text{Linear}$

考虑:

$$X : (B, 4)$$

经过:

$$\text{Linear}(4, 8)$$

得到:

$$Z : (B, 8)$$

经过 ReLU:

$$A = \text{ReLU}(Z) : (B, 8)$$

再经过 Dropout:

$$\tilde{A} = \text{Dropout}(A) : (B, 8)$$

最后:

$$Y = \text{Linear}(8, 2) : (B, 2)$$

Shape 流程为:

$$(B, 4) \rightarrow (B, 8) \rightarrow (B, 8) \rightarrow (B, 8) \rightarrow (B, 2)$$

注意: Dropout 并没有改变维度, 只改变中间数值与本次可用路径。

一个完整训练 Step:

```
1 # 1. 进入训练模式: Dropout 开启
2 model.train()
3
4 # 2. 清掉上一个 Step 累积的梯度
```

```

5 optimizer.zero_grad()
6
7 # 3. Forward: 每次经过 Dropout 都会重新采样随机 Mask
8 pred = model(x_batch)
9
10 # 4. 计算当前随机子网络下的 Loss
11 loss = loss_fn(pred, y_batch)
12
13 # 5. Backward: 梯度通过这一次 Forward 使用的同一个 Mask 回传
14 loss.backward()
15
16 # 6. 根据当前梯度真正更新 Parameter
17 optimizer.step()

```

评估时则是：

```

1 # 1. 切换评估行为: Dropout 关闭
2 model.eval()
3
4 # 2. 关闭梯度记录, 节省推理内存和计算
5 with torch.no_grad():
6     pred = model(x_val)

```

【原理】

Dropout 的随机 Mask 属于 **Forward 时的随机状态**。Backward 不是再随机抽一次 Mask，而是沿着刚才那个 Forward 已经形成的计算图反传。

十八、Dropout 放在 Activation 前还是后？

常见 MLP 写法是：

$$\text{Linear} \rightarrow \text{ReLU/GELU} \rightarrow \text{Dropout}$$

也可以在其他位置看到 Dropout。为什么位置重要？因为随机置零发生在不同语义量上，会改变不同计算。

例如：

$$\text{Dropout}(\text{ReLU}(Wx + b))$$

表示先完成非线性，再随机屏蔽隐藏激活。而：

$$\text{ReLU}(\text{Dropout}(Wx + b))$$

则先对 pre-activation 做随机扰动，再进入 ReLU。二者一般不是同一个函数。

因此科研论文里不能只写：

“we use dropout 0.1”

最好还要说明：

- Dropout 在哪些层；
- 是 hidden dropout、attention dropout 还是 LoRA dropout；
- 是所有 block 共用同一 p ，还是不同位置不同配置。

十九、随机性与可复现性：为什么相同代码可能跑出略不同结果？

Dropout 每个 Forward 都要采样随机 Mask，因此它天然引入新的随机源。

如果希望实验尽量可复现，至少要记录：

- Python / NumPy / PyTorch seed；
- Dataset shuffle seed；
- Dropout probability；
- 模型初始化；
- CUDA / backend 的确定性设置（需要时）；
- 多次独立 seed 的均值和波动。

【常见误区 / 风险】

固定 seed 的目的不是“证明某个结果永远一样”，而是让调试和实验对照更可控。正式科研结论尤其在小数据条件下，最好不要只依赖单个 seed 的偶然结果。

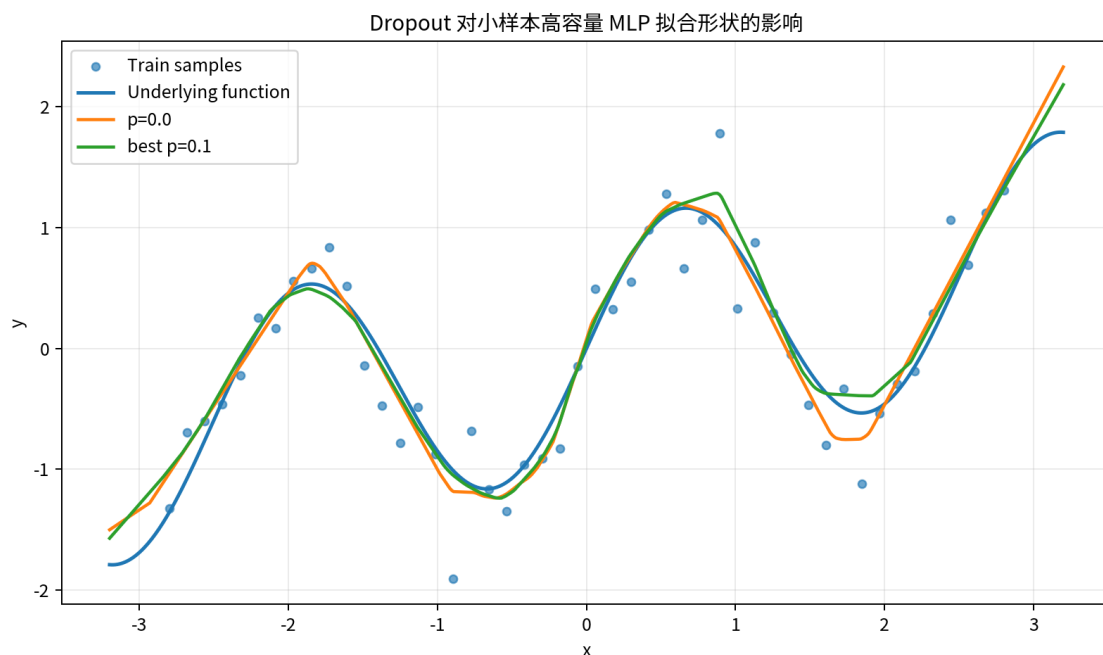


图 4: 同一训练任务中，无 Dropout 与 Validation 选择出的 Dropout 配置的拟合曲线。

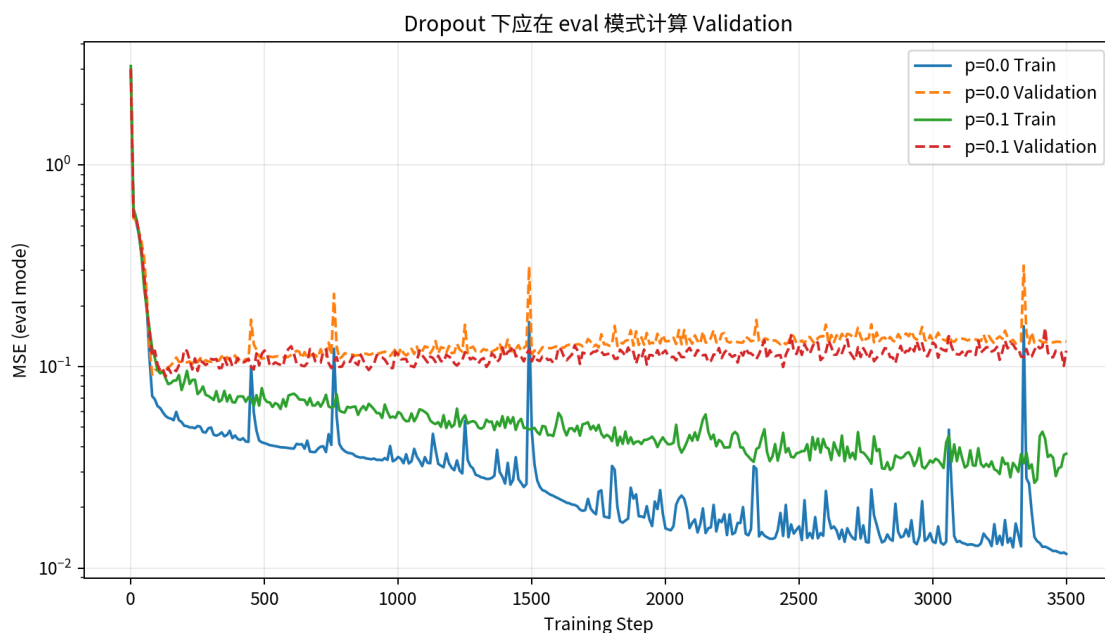


图 5: Train / Validation 曲线应在正确的 train/eval 行为下记录。

二十、MC Dropout: 推理时能不能故意保持 Dropout 开启?

标准推理:

`model.eval()`

Dropout 关闭。

但研究中有一种思路叫 MC Dropout: 在推理时有意保留 Dropout 随机性, 对同一个输入重复 Forward 多次:

$$\hat{y}^{(1)}, \hat{y}^{(2)}, \dots, \hat{y}^{(K)}$$

观察均值和波动, 尝试把输出方差作为某种不确定性信号。

【常见误区 / 风险】

这不是“只要打开 Dropout 就得到可靠 uncertainty”。它是一种具体近似方法, 其校准质量和适用性需要单独验证。对 LLM / Agent Systems 也不能把多次随机输出的方差机械解释为真实风险概率。

二十一、面向科研: Dropout Ablation 应该怎样设计?

如果论文研究 Dropout, 至少应控制:

1. 相同 Dataset split;
2. 相同模型结构;
3. 相同初始化 seed 或多 seed 重复;
4. 相同 Optimizer / LR / Scheduler;

5. 相同 Training Steps / Token Budget;
6. Validation 只用于选择 p 或 checkpoint;
7. Test 只用于最终独立报告;
8. 明确 Dropout 插入位置, 而不只写 “dropout=0.1”。

一个最小 Sweep 可以是:

$$p \in \{0, 0.05, 0.1, 0.2, 0.5\}$$

然后同时报告:

- Train metric;
- Validation metric;
- Test metric;
- 多 seed 均值与波动;
- Training cost / convergence speed。

二十二、与 Agent Systems 的关系

Agent Systems 很多实验并不直接训练大模型, 但 Dropout 知识仍然重要:

- 如果你做 LoRA / QLoRA / 小模型微调, 需要理解 training/eval 行为;
- LoRA 配置本身也可能包含 dropout 参数;
- 如果对 Agent trajectory 做监督微调, 数据量小, 更容易出现 Overfitting;
- 多次随机推理与 “训练态 Dropout” 不是同一概念;
- 评测 Agent 时必须确保被评模型处于预期的 inference mode, 否则随机层会污染可复现性。

【科研洞察】

对 Agent Systems 科研而言, 你当前最需要的是研究 Dropout 本身, 而是能读懂训练 recipe、判断 train/eval 是否正确、识别正则化是否是合理控制变量, 并在微调实验中避免把随机层造成的评测波动误解成 Agent 行为差异。

二十三、常见误区总结

【常见误区 / 风险】

1. “ $p=0.2$ 表示保留 20%。” 错。PyTorch 中 p 是丢弃概率。
2. “Dropout 会永久删除神经元。” 错。只是训练时临时随机 Mask。
3. “训练时置零，推理时再乘 0.8。” 对 PyTorch 默认 inverted dropout 不对；训练时已经缩放，eval 时直接恒等。
4. “no_grad() 会关闭 Dropout。” 错。
5. “Dropout 越大，泛化越好。” 错。过强会欠拟合。
6. “Validation 波动说明模型不稳定。” 未必。先检查评估时是否忘记 `model.eval()`。
7. “Dropout 和 Weight Decay 是同一种 Regularization。” 错。二者作用机制完全不同。

二十四、本讲最终闭环

【本节小结】

1. Dropout 在训练时随机屏蔽 Activation；
2. PyTorch 使用 inverted dropout：保留值乘 $1/(1-p)$ ；
3. 这样训练阶段的输出期望约等于原激活；
4. Dropout 不改变 Shape，也没有可训练 Parameter；
5. train/eval 控制 Dropout 行为，no_grad() 控制 Autograd；
6. Dropout 是随机 Regularization，目标是减少对固定内部路径的脆弱依赖；
7. 太弱可能压不住过拟合，太强则可能造成欠拟合；
8. Transformer 中可以在多个语义位置使用 Dropout；
9. 科研中必须明确 Dropout probability、位置、模式和随机种子。

下一讲：为什么需要 Normalization？——网络越深，Activation 的尺度为什么越来越难控制？